



MINISTRY OF SCIENCE, HIGHER EDUCATION AND INNOVATION
OF THE KYRGYZ REPUBLIC ALA-TOO INTERNATIONAL UNIVERSITY

Faculty of Engineering and Computer Science

COURSEWORK

in the subject programming language 2 (JavaScript)

Topic: "Cafeteria Line Manager Using Java Data Structures"

Completed by: Student of Group COMCEH-24
Aryslanov Akram

Reviewed by: Zhazgul Alymbaeva

Bishkek 2026

INTRODUCTION	3
CHAPTER 1. ANALYTICAL AND THEORETICAL PART ..	4
1.1 Analysis and Study of the Topic	4
1.2 Problem Statement	5
1.3 Methods and Tools Used	6
CHAPTER 2. PRACTICAL PART	8
2.1 System Design	8
2.2 Program Implementation	9
2.3 Testing	11
2.4 Code Example.....	12
CONCLUSION	13
REFERENCES	14
APPENDIX	15

INTRODUCTION

Relevance of the Topic

Data structures are an essential part of software development because they affect program performance and efficiency. Modern applications require fast processing of information and optimized memory usage. Java provides many built-in data structures that simplify development and improve performance.

Goal of the Project

The goal of this coursework is to develop a console-based cafeteria queue management system using Java data structures and analyze the time complexity of operations.

Objectives

- Study Big-O, Big-Ω, and Big-Θ complexity.
- Compare Java data structures.
- Develop a queue management system.
- Implement queue operations using ArrayDeque and HashMap.
- Analyze the efficiency of operations.

Structure of the Coursework

The coursework consists of an introduction, two chapters, conclusion, references, and appendix.

CHAPTER 1. ANALYTICAL AND THEORETICAL PART

1.1 Analysis and Study of the Topic

Example Text

Data structures organize and store information efficiently. Choosing the correct structure improves application performance and reduces execution time.

The project uses:

- ArrayDeque
- HashMap

ArrayDeque is efficient for queue operations because insertion and deletion at both ends work in $O(1)$ amortized time.

HashMap allows fast searching and storing of arrival times with average $O(1)$ complexity.

Big-O Notation

Big-O describes the worst-case complexity of an algorithm.

Examples:

- Array access — $O(1)$
- Linear search — $O(n)$

Big- Ω Notation

Big- Ω describes the best-case complexity.

Example:

- Finding the first element in a list — $\Omega(1)$

Big- Θ Notation

Big- Θ describes the exact asymptotic behavior.

Example:

- Array access — $\Theta(1)$

Complexity Table

Data Structure	Access	Search	Insert	Delete
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
ArrayList	$O(1)$	$O(n)$	$O(1)^*$	$O(n)$
LinkedList	$O(n)$	$O(n)$	$O(1)^{**}$	$O(1)^{**}$
HashSet	-	$O(1)^*$	$O(1)^*$	$O(1)^*$

HashMap	-	$O(1)^*$	$O(1)^*$	$O(1)^*$
ArrayDeque	-	$O(n)$	$O(1)^*$	$O(1)^*$

1.2 Problem Statement

Example Text

University cafeterias often have long queues. Efficient queue management is important for reducing waiting time and improving service quality.

The task of this project is to create a console application that:

- adds students to a queue,
- supports VIP arrivals,
- serves students,
- tracks waiting time,
- calculates average waiting statistics.

The system must work correctly with invalid input and use efficient data structures.

1.3 Methods and Tools Used

Programming Language

Java

Development Tools

- IntelliJ IDEA / VS Code
- Java JDK
- Command Prompt (Windows 10)

Data Structures Used

- ArrayDeque
- HashMap

Methods Used

- Loops
- Conditions
- Static methods
- Console input/output

CHAPTER 2. PRACTICAL PART

2.1 System Design

Example Text

The system is based on a FIFO queue model.

Main Components

Component	Purpose
ArrayDeque	Stores queue
HashMap	Stores arrival time
currentTime	Tracks logical time
totalWait	Stores total waiting time

2.2 Program Implementation

Example Text

The program supports the following commands:

Command	Description
ARRIVE	Add student
VIP_ARRIVE	Add VIP student
SERVE	Serve next student

LEAVE	Remove student
PEEK	Show next student
SIZE	Queue size
PRINT	Print queue
TICK	Increase time
STATS	Show statistics
EXIT	Exit program

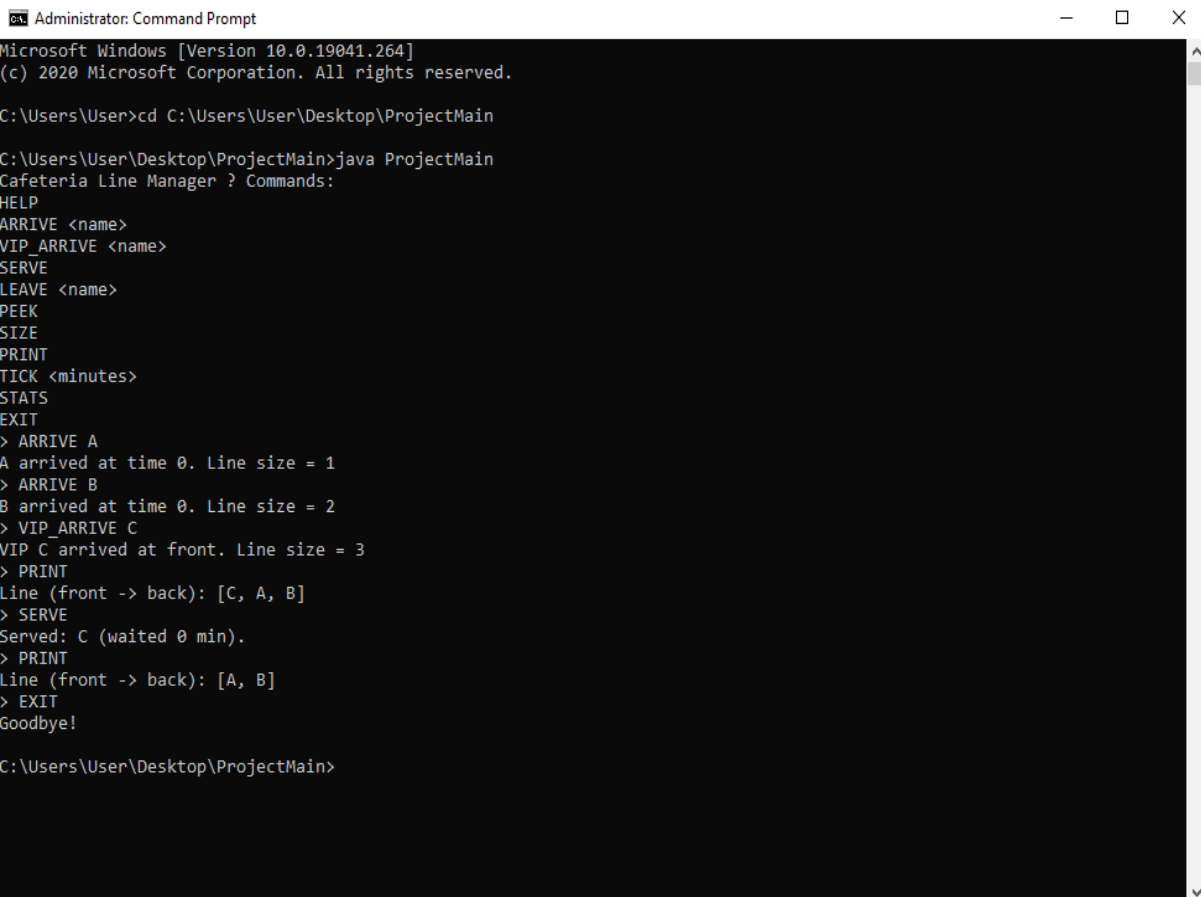
Complexity Analysis

Operation	Complexity
ARRIVE	$O(1)$
VIP_ARRIVE	$O(1)$
SERVE	$O(1)$
LEAVE	$O(n)$
PEEK	$O(1)$

LEAVE requires linear search, therefore complexity is $O(n)$.

2.3 Testing

Example Console Execution



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.19041.264]
(c) 2020 Microsoft Corporation. All rights reserved.

C:\Users\User>cd C:\Users\User\Desktop\ProjectMain

C:\Users\User\Desktop\ProjectMain>java ProjectMain
Cafeteria Line Manager ? Commands:
HELP
ARRIVE <name>
VIP_ARRIVE <name>
SERVE
LEAVE <name>
PEEK
SIZE
PRINT
TICK <minutes>
STATS
EXIT
> ARRIVE A
A arrived at time 0. Line size = 1
> ARRIVE B
B arrived at time 0. Line size = 2
> VIP_ARRIVE C
VIP C arrived at front. Line size = 3
> PRINT
Line (front -> back): [C, A, B]
> SERVE
Served: C (waited 0 min).
> PRINT
Line (front -> back): [A, B]
> EXIT
Goodbye!

C:\Users\User\Desktop\ProjectMain>
```

2.4 Code Example

MAIN DATA STRUCTURES

```
static ArrayDeque<String> line = new ArrayDeque<>();
static HashMap<String, Integer> arrivalTime = new HashMap<>();

static int currentTime = 0;
static long totalWait = 0;
static int servedCount = 0;
```

Explanation:

- ArrayDeque stores the cafeteria queue.
- HashMap stores student arrival times.
- Variables track queue statistics.

ARRIVE COMMAND

```
static void arrive(String name) {
    if (arrivalTime.containsKey(name)) {
        System.out.println("Name already in system");
        return;
    }

    line.addLast(name);
    arrivalTime.put(name, currentTime);

    System.out.println(name + " arrived at time " + currentTime +
        ". Line size = " + line.size());
}
```

Explanation:

- Adds student to the back of the queue.
- Prevents duplicate names.
- Complexity: $O(1)$

VIP ARRIVE

```
static void vipArrive(String name) {  
    if (arrivalTime.containsKey(name)) {  
        System.out.println("Name already in system");  
        return;  
    }  
  
    line.addFirst(name);  
    arrivalTime.put(name, currentTime);  
    System.out.println("VIP " + name + " arrived at front. Line size = " + line.size());  
}
```

Explanation:

- VIP students are inserted at the front.
- Complexity: $O(1)$

SERVE COMMAND

```
static void serve() {  
    if (line.isEmpty()) {  
        System.out.println("No one to serve.");  
        return;  
    }  
  
    String name = line.removeFirst();  
  
    int arrival = arrivalTime.get(name);  
    int wait = currentTime - arrival;  
  
    servedCount++;  
    totalWait += wait;  
  
    arrivalTime.remove(name);  
  
    System.out.println("Served: " + name + " (waited " + wait + " min).");  
}
```

Explanation:

- Removes first student from queue.
- Calculates waiting time.
- Updates statistics.

CONCLUSION

This coursework demonstrated the practical application of Java data structures in solving a real-world problem.

The project successfully implemented a cafeteria queue management system using ArrayDeque and HashMap.

The work improved understanding of:

- asymptotic analysis,
- time complexity,
- Java collections,
- queue-based systems.

The developed application works correctly and demonstrates efficient operation complexity.

REFERENCES

1. [Introduction to Algorithms](#)
2. [Oracle Java Documentation](#)
3. [Algorithms](#)

APPENDIX

Include:

- Full Java source code
- Console screenshots
- Timing experiment screenshots